SILO Land-Use Microsimulation: Models, Mechanisms, and the VAE-SILO Modifications

A technical reference for the VAE → SILO → MITO → MATSim pipeline (Baltimore–Washington MSTM region)

This document describes, in detail, every model inside the SILO (Simple Integrated Land-Use Orchestrator) land-use microsimulation as configured for the Maryland Statewide Transportation Model (MSTM) region, the factors and mechanisms that drive each model, and how a *synthetic base-year population* is evolved year-by-year into a *predicted* future-year population. It then documents the specific code and configuration changes introduced in this project.

Code references point at the working tree `/Users/tomal/Documents/SILO Simulation/silo-master/` — the core engine lives in the `siloCore` module (`siloCore/src/main/java/de/tum/bgu/msm/…`) and the region-specific wiring in the Maryland use case (`useCases/maryland/src/main/java/de/tum/bgu/msm/…`).

1. Where SILO sits in the pipeline

The end-to-end pipeline produces an agent-based travel demand simulation in four stages:

– **VAE (population synthesis).** A conditional variational auto-encoder, trained on ACS PUMS microdata, generates the **synthetic base-year (2016) population** — households, persons, dwellings, and jobs — as CSV files in the SILO input schema. This is SILO's starting state.

– **SILO (land use).** Reads the base-year population and **simulates demographic, labor, income, real-estate, and auto-ownership change one year at a time** from the base year to the end year (here 2016 → 2023), writing a full microdata population for every year.

– **MITO (travel demand).** Consumes a SILO-year population and generates trips / tours (disabled in the runs documented here — `mito.run.travel.model = false`).

– **MATSim (assignment).** Routes the trips on the network (not run here).

SILO is therefore the **temporal engine**: it takes the one-shot VAE cross-section and turns it into a consistent time series of populations, each of which can be validated against the corresponding ACS PUMS 5-year sample.

2. The microsimulation paradigm

SILO is an **event-based microsimulation**. Each simulated year, every model proposes *events* (a birth, a death, a marriage, a job change, a move, …) for individual agents; the `Simulator` collects and applies them, then the year is summarized and (optionally) written out. The core run loop is a single pass over the years:

```
// SiloModel.runYearByYear  (siloCore/.../SiloModel.java)
for (int year = properties.main.startYear; year < properties.main.endYear; year++) {
```

```
    // ... per-year setup (skims, accessibility) ...
    simulator.simulate(year);      // generate + process all events for this year
    dataContainer.endYear(year);   // age the population, adjust income, write per-year output
}
SiloUtil.summarizeMicroData(properties.main.endYear, modelContainer, dataContainer);
```

Three properties govern the horizon and outputs:

Property	Value (this project)	Meaning
`base.year`	2016	Year of the VAE synthetic population (SILO's initial state)
`end.year`	2023	Last simulated year
`household.intermediates.file.ascii` etc.	non-empty (default)	Write full hh/pp/dd/jj microdata **every** year, not just base+end

Because the intermediate-file properties are non-empty, `HouseholdDataManagerImpl.endYear` writes `hh_<year>.csv` / `pp_<year>.csv` (and the dwelling / job managers write `dd_<year>.csv` / `jj_<year>.csv`) for each simulated year into `scenOutput/<scenario.name>/microData/`.

3. Input and output schema (hh / pp / dd / jj)

SILO reads and writes four linked CSV files. The links are: a **person** belongs to a **household** (`hhID`) and may hold a **job** (`workplace` = job `id`); a **household** occupies a **dwelling** (`dd.hhID`); a **dwelling** and a **job** each sit in a **zone**.

```
hh : id, dwelling, hhSize, autos, income, race
pp : id, hhID, age, gender, race, occupation, driversLicense, workplace, income, nationality, relationShip
dd : id, zone, type, hhID, bedrooms, quality, monthlyCost, restriction, yearBuilt
jj : id, zone, personId, type
```

The critical cross-file invariant — central to one of the fixes in §"Changes" — is that `pp.workplace` must equal the `jj.id` of the job that person holds, and that job's `jj.personId` must equal the person's `pp.id`. A vacant job has `personId = -1`.

4. Demographic models

The demographic models simulate the life-events that change *who exists* and *how households are composed*: birth, aging, death, marriage, divorce, children leaving home, and school/work/retirement transitions. Each is invoked once per simulated year and proposes events for eligible agents.

4.1 Birth (BirthModelImpl, DefaultBirthStrategy)

Simulates childbirth for fertile-age women, stochastically adding a newborn person to the mother's household. The annual probability layers a base age/parity rate with marital-status and **per-state** multipliers:

```
P(birth) = localScaler · P_base(age, #children) · maritalStatusScaler · stateFactor
```

`P_base` is the age- and parity-specific base rate from `DefaultBirthStrategy` (ages 15–49, stratified by 0–3+ existing children). Properties: `demographics.localScaler`, `demographics.marriedScaler` / `singleScaler`, `demographics.localScalersFile`, `eventRules.birth`.

```
// BirthModelImpl.java  (lines 95–106)
double birthProb = localScaler * strategy.calculateBirthProbability(
        person.getAge(), HouseholdUtil.getNumberOfChildren(person.getHousehold()));
if (localScalers != null) {
    birthProb *= localScalers.birthFactor(person.getHousehold());   // per-state (DC 0.72)
}
if (person.getRole() == PersonRole.MARRIED) {
    birthProb *= properties.demographics.marriedScaler;
} else {
    birthProb *= properties.demographics.singleScaler;
}
if (random.nextDouble() < birthProb) { giveBirth(person); return true; }
```

4.2 Aging / birthday (BirthdayModelImpl)

Deterministic: every surviving person ages one year, which in turn can trigger occupation transitions downstream (school entry, graduation, retirement — see §4.7).

```
// BirthdayModelImpl.java  (lines 54–62)
private boolean checkBirthday(BirthDayEvent event) {
    Person per = dataContainer.getHouseholdDataManager().getPersonFromId(event.getPersonId());
    if (per == null) return false;     // person died or moved away
    celebrateBirthday(per);            // -> per.birthday()  (age++)
    return true;
}
```

4.3 Death (DeathModelImpl, DefaultDeathStrategy)

Age- and gender-specific mortality, calibrated to US life tables (hardcoded, no property knob). Removes the deceased, handles widow(er) role changes and orphan placement.

```
// DeathModelImpl.java  (lines 43–54)
final Person person = householdDataManager.getPersonFromId(event.getPersonId());
if (person != null) {
    if (random.nextDouble() < strategy.calculateDeathProbability(person)) {  // P(death|age,gender)
        return die(person);
    }
}
```

4.4 Marriage (MarriageModelMstm, DefaultMarriageStrategy)

Pairs eligible (SINGLE/CHILD) persons aged 16+ into co-habiting couples, merges households, finds a dwelling, and assigns car ownership. The Maryland variant adds **race-homophily** matching and **per-state** marriage calibration. Three stages:

– **Intent** — `P(intent) = strategy.calculateMarriageProbability(person) · scale · stateFactor` (`stateFactor` from `LocalDemographicScalers.marriageFactor()`; single-person households get

an extra `onePersonHhMarriageBias`).

– **Matching** — a weighted sampler over the passive pool; in MSTM same-race weight ≈ 10000 vs 0.001 otherwise (tuned by `interracialMarriageShare`).

– **Age-difference** — Gaussian preference: `P(ageDiff) ∝ exp(−(ageDiff+genderBias)²·marryAgeSpreadFac)`.

```
// MarriageModelMstm.java  (lines 240–255)
private double getMarryProb(Person pp) {
    double marryProb = strategy.calculateMarriageProbability(pp) * scale;
    Household hh = pp.getHousehold();
    if (localScalers != null) {
        marryProb *= localScalers.marriageFactor(hh);   // DC 0.65, MD/DE 1.0
    }
    if (hh.getHhSize() == 1) {
        marryProb *= properties.demographics.onePersonHhMarriageBias;
    }
    return marryProb;
}
```

4.5 Divorce (DivorceModelImpl, DefaultDivorceStrategy)

Dissolves MARRIED couples by person-type rate (`P = divorceProbability(personType)/2`; the `/2` because either partner can trigger the single dissolution). Creates a separate household and searches for a vacant dwelling; if none, the divorce is retried next year.

```
// DivorceModelImpl.java  (lines 84–135, condensed)
Person per = householdDataManager.getPersonFromId(perId);
if (per != null && per.getRole() == PersonRole.MARRIED) {
    final double probability = strategy.calculateDivorceProbability(per) / 2;
    if (random.nextDouble() < probability) { /* split household, find dwelling */ return true; }
}
```

4.6 Leave parental household (LeaveParentHhModelImpl)

Moves a CHILD (in a household of ≥2) into an independent SINGLE-person household by a person-type rate, contingent on finding a vacant dwelling.

```
// LeaveParentHhModelImpl.java  (lines 89–100)
if (per != null && qualifiesForParentalHHLeave(per)) {
    final double prob = strategy.calculateLeaveParentsProbability(per);
    if (random.nextDouble() < prob) return leaveHousehold(per);
}
```

4.7 Education / occupation transitions (EducationModelMstm)

The Maryland variant handles **all** age-triggered occupation changes (the generic `EducationModelImpl` only dropped students ≥19 to unemployed). Three transitions, recalibrated to MSTM-region ACS PUMS 2012–16 in 2026:

- **TODDLER** → **STUDENT** at age 5 (deterministic; income 0).
- **STUDENT** → **UNEMPLOYED** ages 18–29 (annual `SCHOOL_EXIT_PROB[age]`: 0.34 at 18 … 0.15 at 25–29, forced at 30 — reproduces ~5.7% still enrolled at 25).
- **EMPLOYED/UNEMPLOYED** → **RETIREE** age 62+ (`RETIREMENT_PROB_*`: employed 0.08 at 62 … 0.30 at 85+; unemployed retire faster). On retirement, income is set to a replacement of

pre-retirement earnings with a Social-Security floor.

```
// EducationModelMstm.java  handleRetirement (lines 285–353, condensed)
double retireProb = isUnemployed ? RETIREMENT_PROB_UNEMPLOYED[ageIdx]
                                 : RETIREMENT_PROB_EMPLOYED[ageIdx];
if (random.nextDouble() < retireProb) {
    int preRetirementIncome = person.getAnnualIncome();
    person.setOccupation(Occupation.RETIREE);
    if (!isUnemployed && person.getJobId() > 0) dataContainer.getJobDataManager().quitJob(true, person);
    int repl = (int) (preRetirementIncome * 0.80 + 0.5);   // VAE-SILO 2026-06-18
    person.setIncome(Math.max(repl, SS_FLOOR));             // SS_FLOOR ~ $15k (2016$)
}
```

4.8 Per-state demographic calibration (LocalDemographicScalers)

A 2026-06-20 VAE-SILO addition that calibrates the **existing** birth and marriage models to ACS without touching the VAE base population. It reads a CSV keyed by state FIPS and applies a `birthFactor` / `marriageFactor` per household by its zone's state. DC (FIPS 11) gets 0.72 / 0.65 (its ACS fertility is ~28% below and married share ~35% below MD/DE); all other states stay at 1.0.

```
// LocalDemographicScalers.java  (lines 41–72, condensed)
TableDataSet t = SiloUtil.readCSVfile(properties.main.baseDirectory + file);
int[] st = t.getColumnAsInt("state");
double[] bf = t.getColumnAsDouble("birthFactor");
double[] mf = t.getColumnAsDouble("marriageFactor");
for (int i = 0; i < st.length; i++) { birthByState.put(st[i], bf[i]); marriageByState.put(st[i], mf[i]); }
logger.info("Per-state birth/marriage scalers ENABLED for states " + birthByState.keySet());
```

> Motivation (quoted from the source, 2026-06-20): "the forecast packed DC's population into too few, too-large households — avg household size drifted 2.05 → 2.37 (ACS 1.94) … DC's ACS fertility is 28% below MD and its married share 35% below, so DC gets birthFactor 0.72 / marriageFactor 0.65. This calibrates the EXISTING models to data — it is not a new model and does not touch the VAE-generated base population."

5. Labor, income, and the job market

This subsystem decides who works, where, and for how much. Three models interact each year: the **EmploymentModel** (hires/fires persons and matches them to jobs), the **JobMarketUpdate** (grows/shrinks the stock of jobs to an exogenous forecast), and the **IncomeAdjustment** (evolves each person's earnings). Together they are the single largest driver of household income — and, indirectly (through the income-sensitive auto-ownership model), of car ownership.

5.1 EmploymentModel (EmploymentModelImpl)

At `setup()` the model measures the **labor-participation share** by age and gender from the base population and smooths it across age (a 5-point moving average). Each year it compares the current employment in every age×gender cell to that target share and converts the gap into a

hire probability (for the unemployed) or a quit probability (for the employed):

```
change      = share[g][age] · (employed + unemployed) − employed
changeRate  =  change / unemployed        if change > 0   // probability to find a job
            =  change / employed          if change < 0   // probability to lose a job
```

Persons then draw against `changeRate` to generate `FIND` or `QUIT` events. A `FIND` is also forced for any person whose occupation is EMPLOYED but who holds no job (`jobId <= 0`) — a 2026-06-13 fix that prevents "employed-but-jobless" persons accumulating from migration/churn.

```
// EmploymentModelImpl.getEventsForCurrentYear (lines 98–114)
if (pp.getOccupation() == Occupation.EMPLOYED && !employed) {
    events.add(new EmploymentEvent(pp.getId(), EmploymentEvent.Type.FIND));   // force a match
    continue;
}
if (changeRate[gen][age] > 0 && !employed) {
    if (random.nextDouble() < changeRate[gen][age]) events.add(new EmploymentEvent(pp.getId(), FIND));
}
if (changeRate[gen][age] < 0 && employed) {
    if (random.nextDouble() < Math.abs(changeRate[gen][age])) events.add(new EmploymentEvent(pp.getId(), QUIT));
}
```

A `FIND` event calls `findJob()` → `JobDataManager.findVacantJob()` and, on success, `takeNewJob()`, which assigns the job and sets the new income. A `QUIT` releases the job and applies a 0.6× income haircut. Properties: `demographics.labor.participation.adjuster`, `householdData.realWageGrowthRate`, `householdData.meanIncomeChange`.

5.2 IncomeAdjustment — annual income evolution

`HouseholdDataManagerImpl.adjustIncome()` runs at `endYear` for every person, building per-quintile gender×age×occupation income distributions and dispatching one `IncomeAdjustment` task per person (on a CPU-bounded thread pool — a 2026-06-15 fix that prevented native-thread exhaustion on the 11.8M-person population). Each occupation has its own rule:

- **Employed — freeze-and-grow** (2026-06-15): keep the worker's *own* current income and grow it by the real-wage factor, with mild noise — preserving the income *distribution* rather than reverting everyone to a cell mean. New entrants (income 0) are anchored to the quintile target.
- **Retiree** — flat-nominal drift with a Social-Security floor (~$15k 2016$); the old −2%/yr decay was removed (2026-06-13) because SS is COLA-indexed.
- **Unemployed** — slow decay (mean −20%/yr). **Student** — usually 0 (30% chance of < $15k).

```
// IncomeAdjustment.selectNewIncome  EMPLOYED branch (lines 125–144)
int currentIncome = person.getAnnualIncome();
if (currentIncome <= 0) {                                  // genuine new entrant
    float target = initialIncomeDistribution[g][a][1] * regionalFactor;   // quintile cell target
    return draw(target, max(MIN_SD, target*SD_FRACTION));
}
double grown = currentIncome * wageGrowthFactor;           // freeze-and-grow (×(1+g))
double sd    = Math.max(MIN_SD, grown * SD_FRACTION * 0.5); // mild idiosyncratic noise
return Math.max(0, (int) (grown + N(0, sd)));
```

> Why this matters for this project: `IncomeAdjustment` is freeze-and-grow and **preserves** income. But `takeNewJob` (below) historically *overwrote* a re-hire's income with the

demographic-cell **mean**. Because SILO re-matches essentially every worker to a (renumbered) job each year, that re-anchoring — not `IncomeAdjustment` — was collapsing the income distribution upward. The fix is in §"Changes".

5.3 New-hire income in takeNewJob

`takeNewJob` sets the income of a person who has just found a job. The base-year code anchored it to `getAverageIncome(gender, age, occupation)` — the **cell mean** — indexed by the real-wage factor, plus a ±$5000 draw:

```
// EmploymentModelImpl.takeNewJob (base-year logic, lines 259–270)
float avgIncome = householdDataManager.getAverageIncome(gender, age, person.getOccupation());
float wageGrowthFactor = (float) Math.pow(1.0 + g, Math.max(0, currentYear - startYear));
final int inc = Math.max((int) (avgIncome * wageGrowthFactor) + change[sel], 0);
person.setIncome(inc);
```

This is the line modified by this project (see §"Changes") to preserve the worker's existing earnings instead of re-anchoring to the mean.

5.4 JobMarketUpdate — growing/shrinking the job stock

Each year `JobMarketUpdateImpl` compares the current number of jobs in every (zone, industry) cell to the **employment forecast** and queues `AddJobsDefinition` (create vacant jobs) or `RemoveJobsDefinition` (remove jobs — vacant first, then occupied via `firePerson`→`quitJob`):

```
// JobMarketUpdateImpl.updateJobInventoryMultiThreadedThisYear (lines 100–121, condensed)
int forecast = (int) jobDataManager.getJobForecast(year, zoneId, jt);
int change = forecast - jobsByZone[JobType.getOrdinal(jt)][zoneId];
if (change > 0) executor.addTaskToQueue(new AddJobsDefinition(zone, change, jt, ...));
else if (change < 0) executor.addTaskToQueue(new RemoveJobsDefinition(zone, -change, jt, vacantJobs, occupiedJobs,
    ...));
```

`RemoveJobsDefinition` removes vacant jobs first and only fires workers if vacancies are insufficient. The mismatch between our base job distribution and the forecast file is therefore what determined how many workers were displaced — central to one of the fixes in §"Changes".

5.5 The employment forecast (JobDataManagerImpl)

`calculateEmploymentForecast()` offers two methods, selected by `job.forecast.method`:

- **INTERPOLATION** — reads an exogenous CSV of (zone × industry) job counts at fixed years and linearly interpolates between them. The forecast targets are independent of the loaded population. (A 2026-06-11 fix sorts the year columns chronologically; previously the MSTM file was interpolated on the wrong segment, "firing ~3.4M workers".)
- **RATE** — counts the *actual* jobs in the synthetic population at the base year, then grows each (zone, industry) cell geometrically by `job.growth.rate`. Because the base-year forecast equals the loaded population, there is **no base-year reshuffling**.

```
// JobDataManagerImpl.calculateEmploymentForecastWithRate (lines 178–198, condensed)
for (Job job : jobData.getJobs())                       // base-year forecast = SP counts
    jobCountBaseyear.get(job.getZoneId()).merge(job.getType(), 1f, Float::sum);
... for each later year:
    count = base * Math.pow(1 + growthRateInPercentByJobType.get(jobType)/100, year - startYear);
```

`findVacantJob()` selects a region for a job-seeker with a `Sampler` weighted by `commutingTimeProbability(travelTime) × numberOfVacantJobs`; if every weight is 0 it falls back to inverse-distance weighting. `identifyVacantJobs()` rebuilds the vacant-job index from scratch each year (2026-06-14 fix — it previously only appended, leaving stale entries). The `findVacantJob` fallback path is where this project fixed an array-overflow crash (§"Changes").

6. Real estate, relocation, and auto ownership

These models evolve *where* households live, the *housing stock* they occupy, *migration* in and out of the region, and *car ownership*.

6.1 Residential moves (MovesModelImpl)

A two-stage model. **Stage 1 (move-or-stay):** a household moves if its current dwelling utility falls below the average satisfaction for its household type, via a logistic probability:

```
P(move) = 1 − 1 / (1 + 0.03 · exp(10 · (avgSatisfaction − currentUtil)))
```

```
// MovesModelImpl.moveOrNot (lines 275–286, condensed)
final double currentUtil   = satisfactionByHousehold.get(household.getId());
final double avgSatisfaction = averageHousingSatisfaction.getOrDefault(hhType, currentUtil);
final double prob = movesStrategy.getMovingProbability(avgSatisfaction, currentUtil);
return random.nextDouble() <= prob;
```

Stage 2 (where-to): a region is chosen by MNL on regional utility and vacancy, then up to `MAX_NUMBER_DWELLINGS = 20` vacant dwellings in that region are scored by a housing-utility strategy and one is sampled. Moves are also what *execute* marriages, divorces, leaving home, demolitions, and in-migration (each searches for a dwelling through this model). Output: `relocation.csv`.

6.2 In/out migration (InOutMigrationImpl)

Drives regional population growth/decline to an exogenous control. Three modes (`population.control.total.method`): **POPULATION** (absolute targets), **MIGRATION** (explicit flows), **RATE** (geometric growth). In-migrants are created by **cloning** existing households (preserving demographic structure) and placing them in vacant dwellings; out-migrants are removed. Recent VAE-SILO additions (2026-06-13/17/19) add optional **per-state** control and **composition-targeted** migration that nudges each state's race and single-person-household marginals toward ACS, with bounded annual churn (≤6%).

```
// InOutMigrationImpl.controlStateComposition (lines 312–361, condensed)
```

```
int target = (int) tblPopulationTargetByState.getIndexedValueAt(year, String.valueOf(st));
double[] marg = marginals.get(year).get(st);          // [white,black,hispanic,other,onePerson]
double[] inW = computeUnderRepresentationWeights(pool, marg);
int churn = (int) Math.min(0.06 * currentPop, 0.6 * gap * currentPop);
weightedOut(pool, inW, max(0, currentPop - target) + churn, events);
weightedIn (pool, inW, max(0, target - currentPop) + churn, events, st);
```

6.3 Construction (ConstructionModelImpl)

Builds new dwellings where demand is high. Per region and dwelling type, demand responds to the **vacancy rate** relative to the type's structural vacancy α (high demand below α, decaying above), and the **location** of new units is chosen by a log-model on `α_type·price + γ_type·accessibility` (denser types weight accessibility more). A one-year lag applies. Properties: `realEstate.constructionLogModelBeta`, `…Inflator`.

```
// ConstructionModelImpl (lines 103–115, condensed)
demandByRegion[dto][region] = demandStrategy.calculateConstructionDemand(
        vacancyByRegion[dto][region], dt, dwellingsByRegion.get(region).size());
// location chosen ∝ exp(beta · (a_type·price(zone) + g_type·accessibility(zone)))
```

6.4 Demolition (DemolitionModelImpl) and Renovation (RenovationModelImpl)

Demolition removes dwellings by a probability that depends on construction era and occupancy — vacant units are ~9× more likely to be razed; occupied units force the resident household to relocate or out-migrate.

```
P(demolish) = vacantModifier · baseRate(yearBuilt)       vacantModifier = 0.9 vacant / 0.1 occupied
```

Renovation moves a dwelling's quality up or down (±1, ±2, or unchanged) each year, with the transition probability rebalanced toward the base-year quality distribution (`base_prob[Δ] · initialShare[target]/currentShare[target]`).

6.5 Pricing and vacancy (PricingModelImpl, RealEstateDataManagerImpl)

Dwelling prices adjust annually to the **regional vacancy rate** of each type relative to its structural rate α, capped at ±10%/yr: vacancy below 0.9α drives prices up steeply, near α prices adjust gently, above 2α prices plateau, and above 10% vacancy prices are frozen.

```
// PricingModelImpl.updateRealEstatePrices (lines 74–100, condensed)
double changeRate = strategy.getPriceChangeRate(vacancyRateAtThisRegion, structuralVacancyRate);
dd.setPrice((int) (dd.getPrice() * changeRate + 0.5));   // changeRate clamped to [0.9, 1.1]
```

6.6 Auto ownership (MaryLandUpdateCarOwnershipModel)

A segmented multinomial-logit for 0/1/2/3+ vehicles. Each alternative's utility combines alternative-specific constants with household size, number of workers, transit accessibility, and income/density category terms:

```
// MaryLandUpdateCarOwnershipModel.expUtilities (lines 111–120)
double one   = Math.exp(ASC_ONE   - 0.121*s + 0.327*w - 0.022*transitAcc + INC_ONE[i]   + DENS_ONE[d]);
double two   = Math.exp(ASC_TWO   + 0.689*s + 0.652*w - 0.051*transitAcc + INC_TWO[i]   + DENS_TWO[d]);
double three = Math.exp(ASC_THREE + 0.801*s + 1.378*w - 0.054*transitAcc + INC_THREE[i] + DENS_THREE[d]);
// P(k) = exp(U_k)·exp(asc_k) / (1 + Σ_j exp(U_j)·exp(asc_j))
```

This model is **income-sensitive** (`INC_*[i]`), which is why the income inflation diagnosed in this project cascaded directly into car-ownership over-prediction. A 2026-06-16 VAE-SILO change activated the model (MSTM previously left new households at zero autos) and re-calibrated its alternative-specific constants per (region × worker-class) cell (with backoff to region then global when a cell has < 300 base households), so the simulated base-year auto distribution matches the observed one and spatial heterogeneity is preserved.

7. Simulation engine, model order, and data flow

Having described the individual models, this section shows how they are orchestrated each year and how data flows from the input CSVs to the per-year output CSVs.

7.1 Entry point and run phases

`SiloMstm.main` loads the properties, builds the data container, reads the input population, assembles the model container, registers result monitors, and runs the model in three phases — `setupModel()`, `runYearByYear()`, `endSimulation()`.

```
// SiloMstm.main (useCases/maryland/.../run/SiloMstm.java, lines 24–44, condensed)
Properties properties = SiloUtil.siloInitialization(args[0]);
DataContainer dataContainer = DataBuilder.buildDataContainer(properties, config);
DataBuilder.readInput(properties, dataContainer);                       // read hh/pp/dd/jj
ModelContainer modelContainer = ModelBuilderMstm.getModelContainerForMstm(dataContainer, properties, config);
SiloModel model = new SiloModel(properties, dataContainer, modelContainer);
model.addResultMonitor(new DefaultResultsMonitor(dataContainer, properties));
model.runModel();
```

7.2 The year loop

```
// SiloModel.runYearByYear (siloCore/.../SiloModel.java, lines 126–156, condensed)
for (int year = properties.main.startYear; year < properties.main.endYear; year++) {
    logger.info("Simulating changes from year " + year + " to year " + (year + 1));
    dataContainer.prepareYear(year);                        // skims/accessibility/vacancy refresh
    SiloUtil.summarizeMicroData(year, modelContainer, dataContainer);
    simulator.simulate(year);                               // generate + process all events
    dataContainer.endYear(year);                            // age, adjust income, WRITE hh/pp/dd/jj_<year>
}
// after the loop: endSimulation() writes the final (endYear) population
```

7.3 Model registration and execution order

`ModelBuilderMstm` wires the models and registers them with the `Simulator`. The annual execution order is: **Birth → Birthday(aging) → Death → Marriage → Divorce → DriversLicense → Education → Employment → LeaveParentHh → JobMarketUpdate → Construction → Demolition → Pricing → Renovation → ConstructionOverwrite → InOutMigration → Moves → (Transport)**. Car ownership is

registered as a model-update listener that runs each year.

```
// ModelBuilderMstm (useCases/maryland/.../run/ModelBuilderMstm.java, lines 152–162)
// VAE-SILO 2026-06-15: MSTM wired NO car-ownership update model, so new / in-migrant
// households kept 0 autos (882k new households at 57.8% zero-car dragged the region 9.2% -> 16.8%).
modelContainer.registerModelUpdateListener(
        new MaryLandUpdateCarOwnershipModel(dataContainer,
                dataContainer.getAccessibility(), properties, SiloUtil.provideNewRandom()));
```

7.4 The Simulator (event-based core)

Each year the `Simulator` collects the events proposed by every model into one list, **shuffles** them (so the order of life-events is randomized across agents), and processes each through its handler:

```
// Simulator.java (lines 76–118, condensed)
public void simulate(int year) { prepareYear(year); processEvents(); finishYear(year); }

private void prepareYear(int year) {
    for (ModelUpdateListener l : modelUpdateListeners) l.prepareYear(year);  // annual models
    for (EventModel<MicroEvent> m : models.values()) {
        m.prepareYear(year);
        events.addAll(m.getEventsForCurrentYear(year));                      // birth, death, find-job, move…
    }
    Collections.shuffle(events, SiloUtil.getRandomObject());
}
private void processEvents() {
    for (MicroEvent e : events) models.get(e.getClass()).handleEvent(e);      // apply each event
}
```

7.5 Readers and writers (the I/O schema)

`DataBuilder.readInput` reads zones, then households, persons, dwellings, jobs from year-suffixed CSVs. The **Maryland** readers/writers extend the core schema with `race` (and household `income`); the writers used at `endYear` are the `…Mstm` variants, so the output carries those columns:

File	Output columns (Maryland writers)
hh	`id, dwelling, hhSize, autos, income, race`
pp	`id, hhID, age, gender, race, occupation, driversLicense, workplace, income, nationality, relationShip`
dd	`id, zone, type, hhID, bedrooms, quality, monthlyCost, restriction, yearBuilt`
jj	`id, zone, personId, type`

```
// HouseholdDataManagerMstm.endYear (useCases/maryland/.../data/HouseholdDataManagerMstm.java, 178–196)
public void endYear(int year) {
    delegate.endYear(year);
    // Patch 2026-04-30: re-write hh + pp with Mstm writers so race + hh.income survive the round-trip.
    new HouseholdWriterMstm(getHouseholds()).writeHouseholds(outDir + "/hh_" + year + ".csv");
    new PersonWriterMstm(this).writePersons(outDir + "/pp_" + year + ".csv");
}
```

Per-year writing is enabled because the `*.intermediates.file.ascii` properties are non-empty; otherwise only the base and end years would be written. Files land in

`scenOutput/<scenario.name>/microData/`.

7.6 Accessibility and travel-time skims

Models that involve location (moves, construction, job search) read zone-to-zone travel times. `AccessibilityImpl.calculateHansenAccessibilities(year)` builds Hansen accessibility from the travel-time skims and the employment/dwelling distribution; travel times come through a `TravelTimesWrapper` backed by `SkimTravelTimes` (reads OMX matrices — the native-HDF5 dependency that required the x86-64 JVM in this project, see §8) or by MATSim when coupled.

```
// AccessibilityImpl.calculateHansenAccessibilities (siloCore/.../AccessibilityImpl.java, 83–100, condensed)
final Map<Integer, List<Job>> jobsByZone = jobData.getJobs().stream().collect(groupingBy(Location::getZoneId));
IndexedDoubleMatrix1D employment = new IndexedDoubleMatrix1D(geoData.getZones().values());
for (int zoneId : geoData.getZones().keySet())
    employment.setIndexed(zoneId, jobsByZone.getOrDefault(zoneId, List.of()).size());
// accessibility(i) = Σ_j employment(j) · f(traveltime(i,j))
```

8. Changes introduced in this project (VAE-SILO drop-in)

This section documents the modifications made so that SILO runs correctly on the **VAE-generated synthetic population** and reproduces the observed year-by-year ACS trajectories. They fall into three groups: (A) environment/build changes that make the run execute at all; (B) source-code and data fixes that correct model behaviour on the VAE population; and (C) the validation framework added around the run. A set of earlier VAE-SILO model calibrations already present in the code tree (dated 2026-06-11 … 06-20) is summarized at the end.

8.A Environment and build changes (make the run execute)

These do not change model logic; they make SILO runnable on the VAE population on this machine.

– **Corrected classpath.** `mvn dependency:build-classpath` resolved a *stale* `.m2` copy of `siloCore` that lacked `EducationModelMstm`. Fix: put the freshly-built local reactor jars first — `maryland.jar : siloCore/target/siloCore.jar : matsim2silo.jar : <deps>`.

– **x86-64 JVM + native HDF5.** SILO reads OMX travel-time skims through the NCSA `jhdf5` JNI library, which is **x86-64 only**; it will not load into an arm64 JVM. Fix: run the x86-64 **JDK 21** (`temurin-21-x64`) and stage `libjhdf5.dylib`/`libjhdf.dylib` on `-Djava.library.path`.

– **Vacant-job buffer.** The VAE `jj` had every job filled (1:1 with workers). SILO's labor market needs vacant jobs (the reference MSTM file is ~27% vacant), otherwise `findVacantJob` floods "No jobs remaining". Fix: append vacant jobs (`personId = -1`) so the stock matches the proven ~8.4M total, keeping the filled-job IDs unchanged so `pp.workplace` links survive.

8.B Source-code and data fixes (correct model behaviour)

8.B.1 `findVacantJob` array-overflow crash (siloCore)

Symptom. The run crashed in year 2018 with `ArrayIndexOutOfBoundsException: Index 31 out of bounds for length 31` in `Sampler.incrementalAdd`, from `JobDataManagerImpl.findVacantJob`.

Cause. The `regionSampler` is allocated with capacity `regions.size()` (31). The first loop adds regions weighted by commuting-time probability. If *all* those weights come out 0 (a job-seeker whose home zone is beyond the commute distribution's support from every region with vacancies), the cumulative probability is 0 and a **fallback loop re-adds** regions to the *same* sampler — but the internal `index` has already advanced, so it overflows.

Fix. Reallocate the sampler before the fallback loop so its index resets:

```
// JobDataManagerImpl.findVacantJob  (the fallback branch)
if (regionSampler.getCumulatedProbability() == 0) {
    // fix: use a FRESH sampler so the incrementalAdd index resets (the first
    // loop already advanced it; re-adding here overflowed the 31-slot array)
    regionSampler = new Sampler<>(regions.size(), Region.class, SiloUtil.getRandomObject());
    for (Region reg : regions) {
        if (getNumberOfVacantJobsByRegion(reg.getId()) > 0) {
            int tt = (int) (travelTimes.getTravelTimeToRegion(homeZone, reg, peakHour, car) + 0.5);
            regionSampler.incrementalAdd(reg, 1.0 / Math.max(1, tt));   // inverse-distance fallback
        }
    }
}
```

With this fix the run completes all years 2016 → 2023.

8.B.2 `pp.workplace` person↔job linkage (data + source)

Symptom. 2.4M "referenced non-existing job" warnings; SILO repeatedly cleared and re-matched workplaces.

Cause. The VAE workplace-allocation step wrote the **work zone** into `pp.workplace` instead of the **job id**. Among 6.09M employed persons there were only ~1,585 distinct workplace values (all in 1–1674, i.e. zone IDs), so the person→job link was effectively random; the job→person link (`jj.personId`) was correct.

Fix (data). Rebuild `pp.workplace` by inverting `jj.personId` (job→person becomes person→job), yielding a perfect bijection (6,091,727 unique job IDs, 100% valid, 100% consistent with `jj.personId`).

Fix (source). Correct the generator so regenerations are right:

```
# vaelib/workplace.py  (assign)  — before:  wp[pid] = z   (the work zone)
wp[int(pid)] = jid                # SILO workplace = JOB id (not zone); job created next line
jj.append((jid, z, int(pid), "job")); jid += 1
```

> Note: this was a genuine data error worth fixing (it broke job-fill and produced millions of spurious warnings), but it was **not** the cause of the income/autos inflation — re-running with

it fixed left those metrics unchanged. The real causes are 8.B.3 and 8.B.4.

8.B.3 Job-market forecast method: `interpolation` → `rate` (properties)

Symptom. A one-year *step* in household income (median bias +14–28%) and autos (mean 1.64 → 1.78) at the first simulated year, then flat — not a gradual drift.

Cause. With `job.forecast.method = interpolation`, `JobMarketUpdate` targets an exogenous MSTM forecast file whose **2016 per-zone/type** job distribution differs from the VAE workplace allocation by **2.35M jobs (28%)** (the *totals* match within 1%; the *spatial* distribution does not). `JobMarketUpdate` "corrected" this in year 2016 by removing surplus jobs — exhausting the vacancy buffer and **firing ~668k workers (11% of the workforce)** — who were then re-hired and re-anchored to the demographic-cell mean income.

Fix. Switch to the **rate** method, whose base-year forecast equals the *actual* SP job counts per zone/type (zero base-year reshuffling), growing uniformly thereafter:

```
# siloMstm_uvae_2023.properties
job.forecast.method      = rate      # was: interpolation
job.growth.rate          = 1.0       # ~1%/yr, matches regional employment + population growth
```

This eliminated the 11% firing (confirmed: 0 forced firings in year 2016).

8.B.4 `takeNewJob` income preservation (siloCore) — the income/autos root cause

Symptom. Even after 8.B.2 and 8.B.3, household income still inflated identically: the employed median jumped +24% in the first two years and froze. A per-person trace showed income is *preserved* in year 2016 (corr 0.997, freeze-and-grow) but **reverts toward the cell mean in 2017** (corr 0.862; the $0–20k bin grows ×1.26, the $150k+ bin shrinks ×0.90 — both pivoting on the cell mean).

Cause. SILO re-matches essentially every worker to a (renumbered) job each year. `takeNewJob` overwrote each re-hire's income with `getAverageIncome(gender, age, occupation)` — the demographic **cell mean**. Because earnings are right-skewed (mean ≈ 36% above median), re-anchoring ~all workers to the mean every year collapses the distribution **upward**, inflating the household median and cascading into the income-sensitive auto-ownership model. (`IncomeAdjustment` itself is correct freeze-and-grow; it is not the culprit.)

Fix. Preserve the worker's existing earnings when they already have some; only genuine new entrants (no prior income) are anchored to the cell target. `IncomeAdjustment` then applies the single real-wage-growth step at year-end.

```
// EmploymentModelImpl.takeNewJob
// BEFORE:
final int inc = Math.max((int) (avgIncome * wageGrowthFactor) + change[sel], 0);   // cell MEAN
person.setIncome(inc);

// AFTER (2026-06-25):
```

```
final int priorIncome = person.getAnnualIncome();
final int inc;
if (priorIncome > 0) {
    inc = priorIncome;                          // preserve; IncomeAdjustment grows it at endYear
} else {
    inc = Math.max((int) (avgIncome * wageGrowthFactor) + change[sel], 0);   // genuine new entrant
}
person.setIncome(inc);
```

Both code fixes (8.B.1, 8.B.4) are compiled with the x86-64 JDK 21 (Java-21 bytecode) and patched into `siloCore-0.1.0-SNAPSHOT.jar` and `siloCore/target/classes`.

8.C Validation framework (added)

- `collect_yearly_output.py` — copies each simulated year's `{hh,pp,dd,jj}_<year>.csv` and the aggregate result files into `Updated SILO/silo_output/<year>/`.
- `validate_by_year_acs.py` — validates every SILO year against that year's **ACS PUMS 5-year** sample (MD/DC/DE; incomes deflated to 2016$ via CPI-U so all vintages are commensurable), producing per-year/per-state figures for household size, autos, dwelling type, income class, age, gender, race, and occupation, plus a `summary.csv`. It reuses the proven recoding from the project's `scripts/05d_validate_silo_by_year_acs.py`.

8.D Earlier VAE-SILO model calibrations already in the tree

These were applied before this session (dated comments in the source) and remain active; they calibrate the *existing* models to ACS without altering the VAE base population:

Date	Component	Change
2026-06-11	JobDataManagerImpl (interpolation)	Sort forecast year-columns chronologically (the MSTM file was interpolated on the wrong segment, "firing ~3.4M workers")
2026-06-12	EducationModelMstm	Recalibrated school-exit (to age 29) and retirement (start 62) hazards to MSTM ACS PUMS
2026-06-13	IncomeAdjustment / EmploymentModel	Real-wage-growth indexation; retiree drift set flat-nominal (COLA), removing –2%/yr decay
2026-06-13/17/19	InOutMigrationImpl	Per-state population control; composition-targeted migration (race + single-person marginals); per-state in-migrant placement
2026-06-14	JobDataManagerImpl	Rebuild the vacant-job index from scratch each year (was appending stale entries)
2026-06-15	IncomeAdjustment; JobMarketUpdate; adjustIncome	Freeze-and-grow employed income; CPU-bounded thread pools (prevent native-thread exhaustion at ~11.8M persons)
2026-06-16	MaryLandUpdateCarOwnershipModel	Activate + re-calibrate auto ownership per (region × worker-class); MSTM previously left new households at 0 autos

Date	Component	Change
2026-06-16/18	EducationModelMstm	Retirement income replacement (0.80 of pre-retirement earnings) with a Social-Security floor (~$15k 2016$)
2026-06-20	LocalDemographicScalers	Per-state birth/marriage calibration (DC 0.72 / 0.65) to fix DC household-size drift

8.E Status and the residual DC drift

The income/autos fixes (8.B.3, 8.B.4) were verified at the per-person level and are being confirmed on a full re-run. The remaining gap is the **DC demographic drift** (household size and age distributions drift upward over the simulated years) — a documented limit of a closed-region microsimulation: DC's real demographics are sustained by constant in-migration of young single adults that a closed model cannot fully reproduce, even with the per-state demographic scalers (8.D, 2026-06-20). MD and DE track ACS closely throughout.